SEARS Customization Guide for AI-Assisted Development

Form Overview

- **Form 1: NewProduct.dialog.js** – A dialog form for creating new product experiments with basic input fields.
- **Form 2: Form.dialog.js** – A comprehensive update form with file uploads, dynamic arrays, and tabbed interface for editing existing product data.

General Architecture Pattern

Both forms follow this structure:

- **State Management:** React hooks for form data, dialog visibility, and file handling
- **Data Source:** MongoDB Atlas functions via user context
- **UI Framework:** PrimeReact components
- **File Handling:** Custom upload components with DigitalOcean storage
- **Validation:** Form submission validation with toast notifications

Customization Guide for NewProduct.dialog.js

1. Adding New Input Fields

Objective: Add new form fields to capture additional experiment data.

> ***AI Prompt Example:***
> *"Add a new input field called 'experiment_duration' to the NewProduct form. It should be a text input that appears after the 'experiment_location' field. Update the emptyProduct object, add the input field to the JSX, and ensure it follows the same validation pattern as other fields."*

Steps:
1. Update emptyProduct object (around line 15):

```
let emptyProduct = {
  // ... existing fields
  experiment_duration: '', // Add new field
  // ... rest of fields
};
```

2. Add input field to JSX (in the dialog content):

```
<div className="field">
  <label htmlFor="experiment_duration">Experiment Duration</label>
  <InputText
    id="experiment_duration"
    value={product.experiment_duration}
    onChange={(e) => onInputChange(e, 'experiment_duration')}
    required
    className={classNames({ 'p-invalid': submitted && !product.experiment_duration })}
  />
</div>
```

3. Update displayProductData function (around line 45) to populate the field:

```
document.getElementById('experiment_duration').value = result[0].experiment_duration;
```

2. Modifying Existing Fields

> ***AI Prompt Example:***
> *"Change the 'polymer_mw' field from a text input to a dropdown with predefined molecular weight options: '10K', '50K', '100K', '500K'. Update both the input component and any related validation."*

Steps:
1. Import Dropdown component:

```
import {Dropdown} from "primereact/dropdown";
```

2. Define options array:

```
const molecularWeightOptions = [
  {label: '10K', value: '10K'},
  {label: '50K', value: '50K'},
  {label: '100K', value: '100K'},
  {label: '500K', value: '500K'}
];
```

3. Replace InputText with Dropdown:

```
<Dropdown
  id="polymer_mw"
  value={product.polymer_mw}
  options={molecularWeightOptions}
  onChange={(e) => onInputChange(e, 'polymer_mw')}
  placeholder="Select Molecular Weight"
/>
```

3. Removing Fields

> ***AI Prompt Example:***
> *"Remove the 'fab_humidity' field completely from the NewProduct form. This includes removing it from the emptyProduct object, the input field from the JSX, and any references in the displayProductData function."*

Steps:

1. Remove from `emptyProduct` object
2. Remove JSX input field
3. Remove from `displayProductData` function
4. Remove any validation references

Customization Guide for Form.dialog.js

4. Adding Dynamic Array Fields

Objective: Add new measurement types that can have multiple entries.

> ***AI Prompt Example:***
> *"Add a new dynamic array field called 'resistance' to the Form.dialog.js. It should have the same structure as other measurement fields (batch, reading, who) and include add/remove functionality. Place it in a new tab called 'Resistance Measurements'."*

Steps:
1. Update emptyProduct structure (around line 20):

```
"resistance": [{
  "batch": "Please enter batch number",
  "reading": "Please enter reading",
  "who": "Please enter your group name"
}]
```

2. Create handler functions:

```
const handleResistanceChange = (e, index, field) => {
  const newResistance = [...formData.resistance];
  if(field === 'batch'){
    newResistance[index].batch = e.target.value;
  }
  if(field === 'reading'){
    newResistance[index].reading = e.target.value;
  }
  if(field === 'who'){
    newResistance[index].who = e.target.value;
  }
  setFormData({ ...formData, resistance: newResistance });
};

const addResistance = () => {
  setFormData({
    ...formData,
    resistance: [...formData.resistance, { batch: '', reading: '', who: '' }],
  });
};
```

3. Add to JSX within TabView:

```
<TabPanel header="Resistance Measurements">
  {formData?.resistance.map((singleResistance, idx) => (
    <div key={idx}>
      <label>Batch ID</label>
      <InputTextarea
        value={singleResistance.batch}
        onChange={(e) => handleResistanceChange(e, idx, 'batch')}
        rows={1}
        cols={30}
      />
      <label>Reading</label>
      <InputTextarea
        value={singleResistance.reading}
        onChange={(e) => handleResistanceChange(e, idx, 'reading')}
        rows={1}
        cols={30}
      />
      <label>Entry Person</label>
      <InputTextarea
        value={singleResistance.who}
        onChange={(e) => handleResistanceChange(e, idx, 'who')}
        rows={1}
        cols={30}
      />
    </div>
  ))}
  <Button label="Add Resistance Measurement" onClick={addResistance} />
  <Button label="Save" onClick={() => saveProduct(allTypes[0])} />
</TabPanel>
```

5. Adding File Upload Capabilities

> ***AI Prompt Example:***
> *"Add file upload capability for a new measurement type called 'xrd_files'. Create the upload component, file display section, and integrate it with the existing file upload system. Include CSV dialog functionality for CSV files."*

Steps:
1. Update allTypes array:

```
const allTypes = [
  "uv_vis_nir_files",
  "iv_files",
  "profilometry_files",
  "giwaxs_files",
  "skpm_files",
  "xrd_files" // Add new type
]
```

2. Add to emptyProduct:

```
xrd_files : [],
"xrd": [{
  "batch": "Please enter batch number",
  "reading": "Please enter reading",
  "who": "Please enter your group name",
  "file": "Please enter file name"
}]
```

3. Create handler functions (similar to other file types):

```
const handleXRDChange = (e, index, field) => {
  const newXRD = [...formData.xrd];
  // Implementation similar to other handlers
  setFormData({ ...formData, xrd: newXRD });
};

const addXRD = () => {
  setFormData({
    ...formData,
    xrd: [...formData.xrd, { batch: '', reading: '', who: '', file: '' }],
  });
};
```

4. Add TabPanel with upload functionality:

```
<TabPanel header="XRD">
  {formData?.xrd.map((singleXrd, idx) => (
    // Form fields similar to other measurement types
  ))}

  <UploadComponent
    label="Upload XRD Files"
    setFile={functionToSetFiles}
    type="xrd_files"
  />

  <div>
    <strong>Uploaded Files</strong>
    {formData?.["xrd_files"]?.map(val => (
      <div key={val?.name}>
        <a href="#" onClick={(e) => handleFileNameClick(val?.name, e)}>
          {val?.name}
        </a>
        {val?.name.endsWith('.csv') && (
          <CsvDialogButton
            fileName={val?.name}
            experimentId={formData?._id?.toHexString()}
          />
        )}
      </div>
    ))}
  </div>

  <Button
    label="Save"
    onClick={() => saveProduct(allTypes[5])} // Use appropriate index
  />
</TabPanel>
```

6. Modifying Existing Tabs

> ***AI Prompt Example:***
> *"Modify the 'Thickness' tab to include an additional field called 'measurement_method' with options 'Profilometer', 'Ellipsometry', 'AFM'. Make it a dropdown selection for each thickness measurement entry."*

Steps:
1. Update data structure:

```
"thickness": [{
  "batch": "Please enter batch number",
  "reading": "Please enter reading",
  "who": "Please enter your group name",
  "measurement_method": "" // Add new field
}]
```

2. Update handler function:

```
const handleThicknessChange = (e, index, field) => {
  const newThickness = [...formData.thickness];
  // Add new case
  if(field === 'measurement_method'){
    newThickness[index].measurement_method = e.target.value;
  }
  // ... existing cases
  setFormData({ ...formData, thickness: newThickness });
};
```

3. Add dropdown to JSX:

```
<label>Measurement Method</label>
<Dropdown
  value={singleThickness.measurement_method}
  options={[
    {label: 'Profilometer', value: 'Profilometer'},
    {label: 'Ellipsometry', value: 'Ellipsometry'},
    {label: 'AFM', value: 'AFM'}
  ]}
  onChange={(e) => handleThicknessChange(e, idx, 'measurement_method')}
  placeholder="Select Method"
/>
```

7. Removing Components

> ***AI Prompt Example:***
> *"Remove the entire 'SKPM' tab and all related functionality from Form.dialog.js. This includes the tab panel, handler functions, data structures, and file upload capabilities."*

Steps:

1. **Remove from allTypes:** Remove "skpm_files"
2. **Remove from emptyProduct:** Remove skpm arrays and objects
3. **Remove handler functions:** `handleSKPMChange`, `addSKPM`
4. **Remove TabPanel:** Delete entire SKPM TabPanel JSX block
5. **Update saveProduct calls:** Adjust array indices if needed

AI Prompting Best Practices

Effective Prompt Structure:

1. **Be Specific:** "Add a dropdown field for 'sample_type' with options A, B, C to the NewProduct form"
2. **Include Context:** "Following the same pattern as the polymer_name field..."
3. **Specify Location:** "Place it after the experiment_location field in the dialog"
4. **Request Complete Implementation:** "Update all necessary parts including state, handlers, and validation"

> **Example Comprehensive Prompt:**
>
> "I need to add a new measurement type called 'photoluminescence' to Form.dialog.js. It should:
> 1. Have fields for batch, reading, who, excitation_wavelength, and emission_wavelength
> 2. Support file uploads with CSV dialog functionality
> 3. Follow the same pattern as the UV-Vis-NIR tab
> 4. Be placed as the last tab in the TabView
> 5. Include proper error handling and toast notifications
> Please update all necessary parts of the code including the data structures, handler functions, JSX, and file upload integration."

Common Patterns to Reference

Data Flow Pattern:

1. **State Management:** `formData` → handler functions → `setFormData`
2. **File Upload:** `UploadComponent` → `functionToSetFiles` → `uploadFile` → `saveProduct`
3. **Validation:** Submit validation → toast notifications → MongoDB update

Component Structure:

```
Dialog
├── TabView
│   ├── TabPanel (Metadata)
│   ├── TabPanel (Measurements with dynamic arrays)
│   └── TabPanel (File uploads)
├── Toast (notifications)
└── Footer (action buttons)
```

This guide provides a foundation for AI-assisted customization of both React forms. Each modification should follow the established patterns for consistency and maintainability.